

В отношении селектора `:not()` действуют несколько ограничений.

- С селектором `:not()` можно использовать только *простые селекторы*. Другими словами, можно применять селекторы тегов (такие как `html` или `p`), универсальный селектор (`*`), классы (например, `.footer`), идентификаторы (например, `#banner`) или псевдоклассы (`:hover`, `:checked`, `:first-child` и т. д.). Таким образом, все следующие селекторы можно считать правильными:

```
.footnote:not(div)
img:not(.portrait)
div:not(#banner)
li:not(:first-child)
```

- Нельзя использовать селекторы потомков (такие как `div p a`), псевдоэлементы (такие как `::first-line`), групповые селекторы или комбинации (такие как родственный смежный селектор `h2 + p`).
- Нельзя в одной строке указывать несколько селекторов `:not()`. Например, следующий код будет неправильным:

```
a[href^="http://"]:not([href*="google.com"]):not([href="yahoo.com"])
```

Другими словами, значение `:not()` в селекторе можно использовать только один раз.

Практикум: использование селекторов

В оставшейся части этой главы вы потренируетесь в создании разных селекторов и увидите, как они влияют на дизайн веб-страницы. Практикум начнем с представления основных типов, а затем перейдем к более современным стилям.

Чтобы начать обучение, вы должны иметь в распоряжении файлы с учебным материалом. Для этого нужно загрузить файлы для выполнения заданий практикума, расположенные по адресу github.com/mrightman/css_4e. Перейдите по ссылке и загрузите ZIP-архив с файлами (нажав кнопку **Download ZIP** в правом нижнем углу страницы). Файлы текущего практикума находятся в папке **03**.

ВОЗМУЩЕННОЕ ЧАВО

Используйте внутренние таблицы стилей

Почему в этом практикуме мы пользуемся внутренними таблицами стилей? Ведь в главе 2 книги рекомендуется применять внешние!

Да, внешние каскадные таблицы стилей используются для создания быстро загружаемых, «производительных» сайтов. Однако внутренние таблицы стилей упрощают разработку одиночных веб-страниц, таких, как в этом практикуме. В данном случае гораздо удобнее работать с одним файлом, вместо того чтобы переключаться между файлом внешней таблицы стилей и веб-страницей.

Кроме того, вы можете пользоваться предварительным просмотром результатов своей работы без постоянного обновления кэша браузера.

Ввиду вышесказанного я рекомендую использовать внешние таблицы стилей для сайтов. Если вы собираетесь использовать стили, созданные в этом уроке, в дальнейшем, а не только в учебных целях — на здоровье. Но в рамках текущего упражнения, чтобы выполнять задания быстро и легко, вы будете пользоваться одиночным HTML-файлом и внутренней таблицей стилей.

1. Откройте файл `selector_basics.html`, расположенный в папке **03**, в редакторе HTML-кода.

Страница состоит из основных HTML-элементов (рис. 3.10). В этом практикуме мы попытаемся придать ей изящный вид. Сначала вы свяжете страницу со шрифтом Google, который уже использовали в предыдущей главе.

2. Введите следующий код в пустой строке сразу после элемента `</title>`:

```
<link href='http://fonts.googleapis.com/css?family=Kurale' rel='stylesheet'>
```

Этот код связывается с внешней таблицей стилей, размещенной на сервере Google. Она загружает шрифт Kurale и тем самым дает вам возможность использовать его на своей странице (подробнее об использовании веб-шрифтов читайте в разделе «Использование веб-шрифтов» главы 6). Теперь необходимо добавить внутреннюю таблицу стилей.

3. Сразу после добавленного на предыдущем шаге элемента `link` нажмите клавишу **Enter** для перехода на новую строку и введите тег `<style>`. Дважды нажмите клавишу **Enter** и введите `</ >`.

Эти открывающий и закрывающий теги элемента `style` нужны для размещения внутренней таблицы стилей. Очень полезно набирать их сразу парой, чтобы случайно не забыть о закрывающем теге. Теги сообщают браузеру, что между ними находятся команды языка CSS. HTML-код теперь должен выглядеть следующим образом (код, который вы только что добавили, выделен полужирным шрифтом):

```
<title>Селекторы</title>
<link href='http://fonts.googleapis.com/css?family=Kurale' rel='stylesheet'>
<style>

</style>
```

Теперь нужно добавить селектор тега (если вы выполнили все упражнения практикума в прошлой главе, то уже умеете это делать).

4. Перейдите к строке между открывающим и закрывающим тегами элемента `style`, а затем наберите код `body {`. Дважды нажмите клавишу **Enter** и введите `}`.

Как я уже отмечал, желательно указывать закрывающую скобку сразу же, как только вы добавили открывающую. Чтобы создать селектор тега, просто укажите имя HTML-элемента, который следует отформатировать. Он будет применен ко всем абзацам текста, заключенным в элемент `body`. Теперь необходимо изменить фоновый цвет и поле вокруг страницы.

5. Щелкните кнопкой мыши между скобками `{` и `}` стиля `body` и добавьте три правила форматирования — цвет, отступ и поле:

```
body {
  background-color: rgb(50,122,167);
  padding: 0 20px 20px 20px;
  margin: 0;
}
```

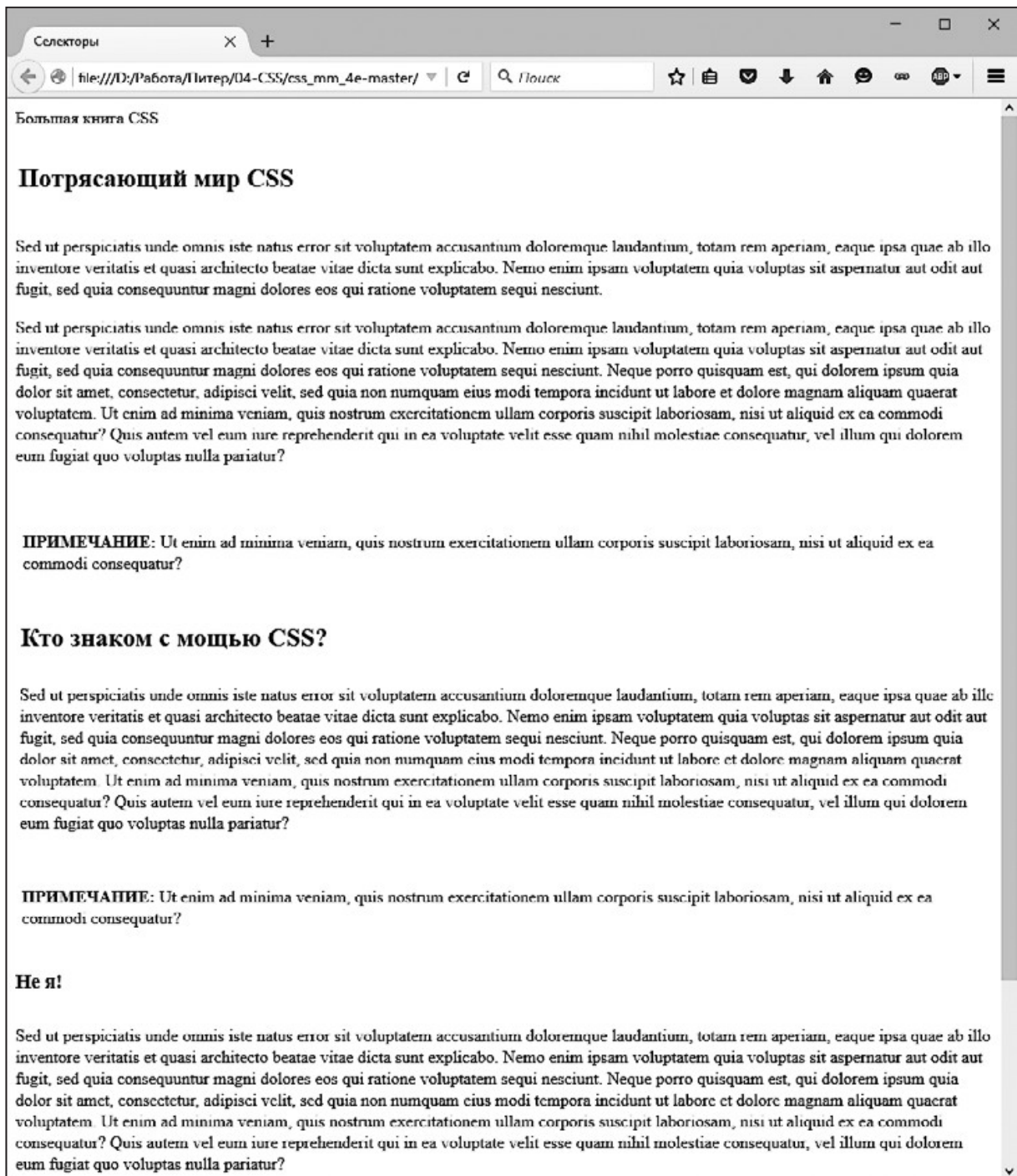


Рис. 3.10. Простой HTML-текст смотрится в браузере чересчур аскетично. Но с помощью каскадных таблиц стилей вы можете из серой страницы, показанной на этом рисунке, получить потрясающе красивую (рис. 3.11), выполнив три десятка простых шагов

Нажимайте клавишу **Enter**, чтобы поместить каждое CSS-свойство на отдельную строку. Кроме того, полезно сделать отступы клавишей **Tab**, чтобы улучшить визуальное восприятие кода CSS (некоторые разработчики вместо табуляции используют два пробела, а вы можете выбрать то, что вам больше нравится).

Свойство цвета фона, которое мы только что изменили, обозначается как `rgb()` — это один из способов указать значения красного, зеленого и синего цветов. В дан-

ном случае цвет фона темно-синий. Он затрудняет чтение текста, поэтому необходимо изменить цвет текста элементов абзаца.

ПРИМЕЧАНИЕ

Если вы новичок в веб-дизайне, то имена свойств и их значения вам пока незнакомы. Так что просто набирайте их в том виде, как показано в практических шагах. Обо всех этих свойствах вы узнаете в главе 6.

6. Добавьте другой стиль под только что созданным стилем `body`.

```
p {  
  color: rgba(255,255,255,.6);  
  font-size: 1em;  
  font-family: "Kurale", Arial, Helvetica, sans-serif;  
}
```

С помощью трех свойств CSS мы изменили форматирование всех абзацев (всех элементов `p`) — определили цвет, размер и шрифт текста. На этот раз, чтобы указать цвет, мы использовали значение `rgba()`. Дополнительная буква `a` в свойстве `rgb` позволяет создать частично прозрачный цвет. В данном примере для абзаца мы использовали белый цвет (его значения — 255, 255, 255) с непрозрачностью 60 % (значение `.6`). Сквозь текст проступает синий цвет, поэтому он кажется светло-синим.

Самое время посмотреть, что у нас получилось.

7. Откройте страницу в браузере для предварительного просмотра.

До изменения настроек текст веб-страницы отображался шрифтом с засечками (таким как Times New Roman). Теперь, если каскадные таблицы стилей функционируют должным образом, вы увидите семь абзацев, для которых задан светло-синий цвет текста и шрифт Kurale.

Создание группового селектора

Нередко бывает так, что несколько различных элементов веб-страницы должны иметь одинаковый внешний вид. Вероятно, и вы хотите, чтобы все заголовки отображались шрифтом одного вида и цвета. Вместо того чтобы создавать отдельные стили и дублировать одни и те же атрибуты и параметры для каждого элемента `h1`, `h2` и т. д., вы можете собрать и сгруппировать несколько элементов в единственный селектор.

1. Вернитесь к своему HTML-редактору с открытым файлом `selector_basics.html`. Сейчас мы добавим новый стиль сразу после только что созданного стиля элемента `p`.
2. Щелкните кнопкой мыши после закрывающей фигурной скобки селектора тега `p`, нажмите клавишу **Enter** для начала новой строки и наберите код:

```
h1, h2, h3 {  
  
}
```

Как описано в этой главе ранее, групповой селектор — это просто список. Данный стиль создает одинаковое форматирование элементов h1, h2 и h3 веб-страницы.

- Щелкните на пустой строке между открывающей { и закрывающей } фигурными скобками и добавьте пять CSS-свойств:

```
color: rgb(255,255,255);
font-family: Arial, "Palatino Linotype", Times, serif;
border-bottom: 2px solid rgb(87,185,178);
padding-top: 10px;
padding-bottom: 5px;
```

Здесь вы задаете цвет и тип шрифта для заголовков, добавляете линию границы под заголовками, устанавливаете отступы снизу и сверху, используя свойство padding. Оно добавляет дополнительное пространство от краев элемента без воздействия на фон или рамку, то есть вы добавляете немного пространства под заголовком и между нижней границей текста и рамкой под ним.

- Сохраните файл и просмотрите его работу в браузере.

Заголовок h1 в начале веб-страницы и заголовки h2 и h3 ниже на странице имеют одинаковое начертание и цвет шрифта, а также зеленую рамку вверх (см. рис. 3.11). Элемент h1 выглядит немного мелковатым, но мы легко можем увеличить его.

- Вернитесь к файлу selector_basics.html в текстовом редакторе. Под только что созданным групповым селектором добавьте еще один стиль:

```
h1 {
  font-size: 2em;
}
```

Этот стиль увеличит размер шрифта. Em — это размер шрифта, который используется в браузерах по умолчанию, поэтому запись 2em означает двойной стандартный размер шрифта. Обратите внимание, как удобно применять сразу несколько стилей к одному элементу: правило h1, h2, h3 и h1 в данном случае. В этом примере групповой селектор и новый селектор тега применяются к элементу h1. Такой процесс в языке CSS называется *каскадом*. Подробнее об этом вы прочитаете в главе 5.

Создание и применение идентификаторов

Идентификаторы предназначены для изменения стиля определенного элемента. Вы создаете стиль и добавляете атрибут id к открывающему HTML-тегу на странице, а затем применяете свойства созданного стиля к этому одиночному элементу. Часто идентификаторы используются для определения элементов формы, создания ссылок на страницах (см. врезку «HTML-элементы div и span» в данной главе), а также помогают применять JavaScript-сценарии для управления элементами на странице. Хотя многие веб-дизайнеры в настоящее время стараются не использовать идентификаторы (почему, вы узнаете в разделе «Управление каскадностью» главы 5), желательно знать, как с ними работать.

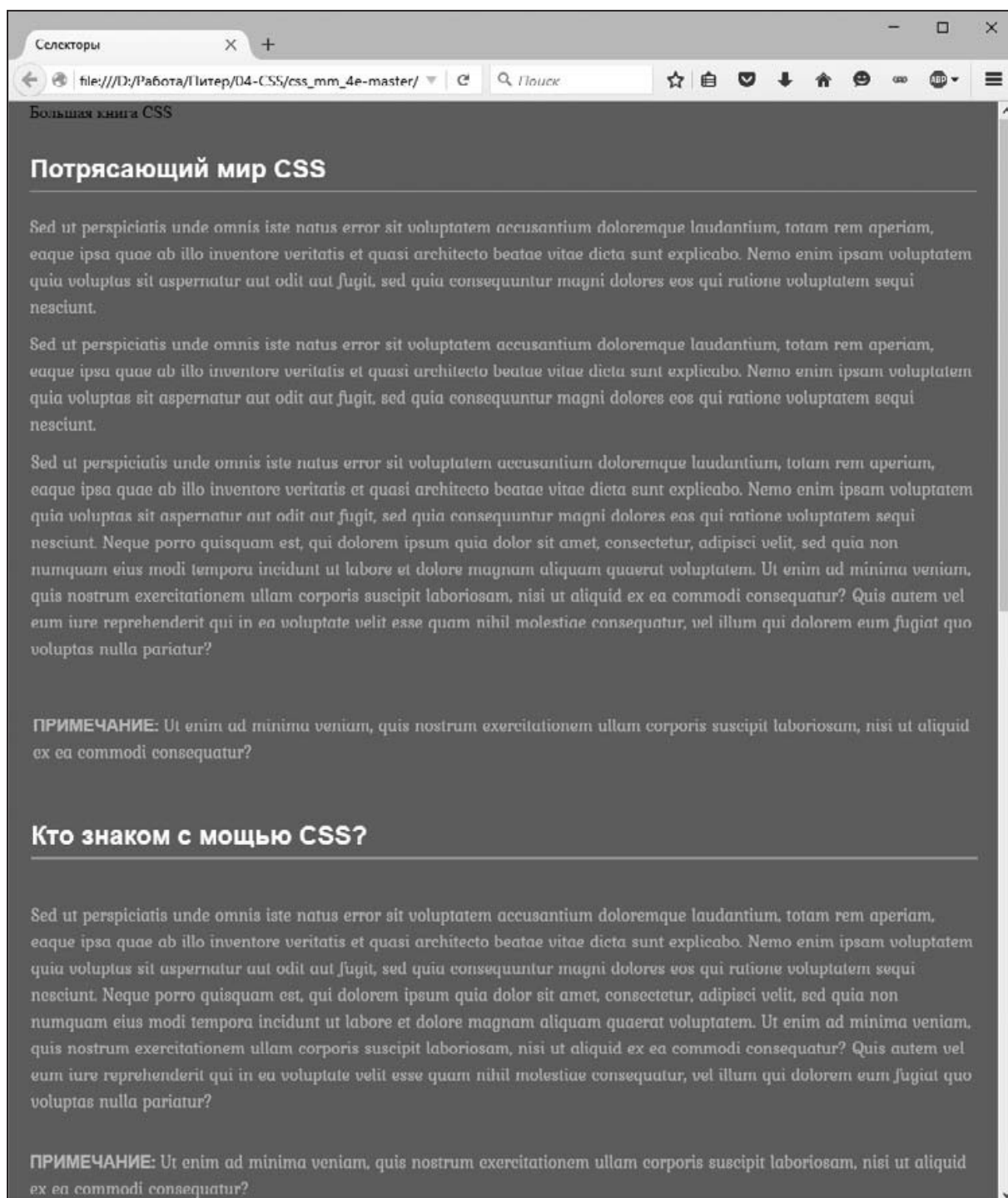


Рис. 3.11. Простой селектор тега может коренным образом преобразовать внешний вид всех вхождений элемента, выполнив форматирование текста веб-страницы. В данном случае групповой селектор делает даже больше, изменяя формат каждого экземпляра заголовков трех различных уровней

В текущем примере мы создадим стиль, который определяет вид текста **Большая книга CSS** (CSS: The Missing Manual) в самом верху страницы. Этот текст играет роль логотипа, и вы создадите специальный идентификатор для его форматирования.

1. Вернитесь к HTML-редактору с открытым файлом `selector_basics.html`.
Добавим после последнего созданного класса `h1` новый стиль.

- Щелкните кнопкой мыши после закрывающей фигурной скобки предыдущего стиля, нажмите клавишу **Enter** для создания новой строки и введите код `#logo {`.

Идентификаторы всегда начинаются с символа решетки (`#`). Имя стиля указывает на тип элемента страницы, который является логотипом сайта.

- Нажмите клавишу **Enter** еще раз и введите следующий код:

```
font-family: Baskerville, Palatino, sans-serif;
font-size: 2em;
color: rgba(255,255,255,.8);
font-style: italic;
text-align: center;
margin-bottom: 30px;
background-color: rgb(191,91,116);
border-radius: 0 0 10px 10px;
padding: 10px;
```

Этот код похож на длинный список свойств, но он лишь устанавливает некоторые свойства шрифта, фоновый цвет страницы и добавляет отступы для текста логотипа.

- Завершите определение стиля, введя закрывающую фигурную скобку. Код стиля полностью должен выглядеть так:

```
#logo {
  font-family: Baskerville, Palatino, sans-serif;
  font-size: 2em;
  color: rgba(255,255,255,.8);
  font-style: italic;
  text-align: center;
  margin-bottom: 30px;
  background-color: rgb(191,91,116);
  border-radius: 0 0 10px 10px;
  padding: 10px;
}
```

Если вы сохраните файл и просмотрите его в браузере, то не увидите никаких изменений. Это обусловлено тем, что данный стиль не делает *ничего*, пока вы его не примените. Для этого нужно добавить атрибут `id` к HTML-коду веб-страницы, обозначая фрагмент, который следует отформатировать.

- Найдите на веб-странице открывающий тег `<div>` с текстом Большая книга CSS. Он расположен после тега `<article>`. Добавьте в тег `<div>` код `id="logo"` следующим образом:

```
<div id="logo">
  Большая книга CSS
</div>
```

Теперь элемент `div` будет отформатирован в соответствии со стилями `#logo`. Как это часто случается при работе с CSS, существует много способов добиться одного и того же результата. Вы могли бы использовать класс и применить его к элементу `div`. Но в данном случае вы используете идентификатор, по-

скольку назначение стиля — идентификация логотипа на странице — соответствует предназначению идентификаторов.

6. Сохраните страницу и просмотрите ее в браузере.

Теперь текст **Большая книга CSS** (CSS: The Missing Manual) находится в небольшом прямоугольнике в верхней части страницы, располагается по центру и выделен цветом (рис. 3.12).



Рис. 3.12. Идентификаторы не единственный способ форматирования отдельных элементов, таких как логотип в верхней части страницы

Создание и применение классов

Селекторы тегов выполняют свои функции быстро и эффективно, но они, можно сказать, совсем неразборчивы в деталях. Что же делать, если вы хотите отформатировать один элемент р на веб-странице иным способом, чем все остальные такие элементы? Решение проблемы — использование классов.

1. Вернитесь к текстовому редактору с открытым файлом `selector_basics.html`.

Добавьте вслед за последним созданным стилем еще один.

2. Щелкните кнопкой мыши после закрывающей фигурной скобки селектора `#logo`, нажмите клавишу **Enter** и введите код:

```
.note {  
  
}
```

Имя `note` (примечание) для стиля выбрано не случайно. Оно соответствует его функциям: стиль выделяет абзацы, содержащие дополнительные примечания для посетителей вашего сайта. Создав класс один раз, вы можете применить его ко всем примечаниям веб-страницы (сайта), например к третьему абзацу.

3. Щелкните на пустой строке между открывающей `{` и закрывающей `}` фигурными скобками и добавьте к стилю следующий перечень свойств:

```
color: black;  
border: 2px solid white;  
background-color: rgb(69,189,102);  
margin-top: 25px;  
margin-bottom: 35px;  
padding: 20px;
```

Обратите внимание, что для определения цвета шрифта и границ вы не использовали параметр `rgb()`. В языке CSS существует несколько способов определения цвета, включая ключевые слова, такие как `white` (белый), `black` (черный) или `orange` (оранжевый). Вы узнаете об этом подробнее в разделе «Форматирование текста цветом» главы 6.

Когда вы просмотрите эту веб-страницу, вы не увидите изменений. Как и в случае с идентификаторами, классы не возымеют никакого эффекта на веб-странице, пока стиль не будет применен к HTML-коду.

4. В HTML-коде веб-страницы найдите абзац `p`, текст которого начинается со слова **ПРИМЕЧАНИЕ**, окруженного тегами элемента `strong`.

Чтобы применить класс к этому элементу, добавьте атрибут `class`, за которым должно следовать имя класса, в данном случае `note`.

5. Щелкните кнопкой мыши сразу за именем открывающего тега `p`, нажмите клавишу **Пробел**, а затем введите код `class="note"`. HTML-код теперь должен иметь такой вид (только что набранный код отмечен полужирным шрифтом):

```
<p class="note"><strong>ПРИМЕЧАНИЕ:</strong>
```

Убедитесь, что *не* ввели атрибут с точкой: `class=".note"`. Точка требуется только во время определения имени класса в таблице стилей; в HTML-коде она не нуж-

на. Повторите этот шаг для второго абзаца (он расположен перед элементом h3 с текстом Не я!).

ПРИМЕЧАНИЕ

Несмотря на имя, которое вы присвоили классу, вы можете применить его к любым другим элементам, а не только к p. Если данное форматирование относится, например, к заголовку h2, то HTML-код должен выглядеть следующим образом:

```
<h2 class="note">
```

6. Сохраните файл и просмотрите веб-страницу в браузере.

Абзац с примечанием красиво подсвечен на странице (рис. 3.13).

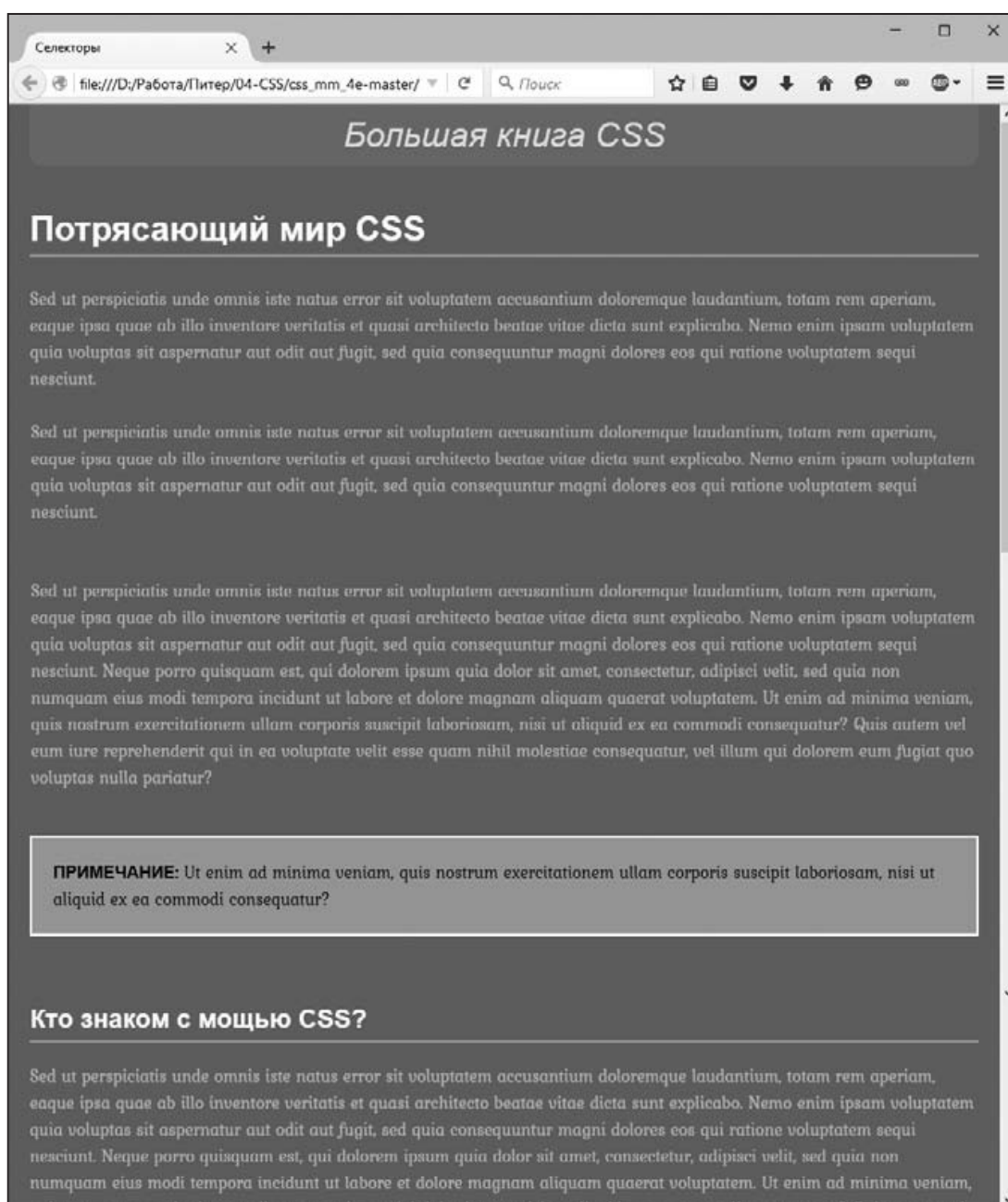


Рис. 3.13. С помощью классов вы можете выполнить ювелирное форматирование элементов веб-страницы

ПРИМЕЧАНИЕ

Если ваша веб-страница не похожа на изображенную на рис. 3.13, возможно, вы указали имя какого-либо свойства или его значение с ошибкой. Проверьте код по шагам. Кроме того, удостоверьтесь в том, чтобы каждая пара «свойство: значение» была завершена точкой с запятой и в самом конце определения стиля присутствовала закрывающая фигурная скобка. Если ваш стиль не работает должным образом, то, скорее всего, в нем не хватает именно этих символов. Это самая распространенная ошибка.

Создание селекторов потомков

На странице `selectors_basics.html` вы применили класс `note` к двум абзацам. Каждый из них начинается словом **Примечание:**, выделенным полужирным начертанием (на самом деле это слово находится внутри элемента `strong`). Но что делать, если вы хотите отформатировать эти слова еще и ярко-оранжевым цветом? Вы могли бы создать стиль для элемента `strong`, но он затронет все эти элементы на странице, в то время как вы хотите изменить только те, которые находятся внутри примечаний. Одним из решений было бы создание класса, например `.noteText`, и применение его к каждому из элементов `strong` внутри примечаний. Но вы наверняка забудете применить класс, если у вас много таких страниц с примечаниями.

Лучший способ решить эту проблему — создать селектор потомков, который относится только к нужным нам элементам `strong`. К счастью, сделать это совсем не сложно.

1. Вернитесь к HTML-редактору и файлу `selector_basics.html`. Создайте новую строку для указания стиля с селектором потомков.

Щелкните кнопкой мыши после закрывающей фигурной скобки стиля `.note` и нажмите клавишу **Enter**.

2. Введите код `.note strong {`.

Последний слово селектора — `strong` — это и есть элемент, который вы хотите отформатировать. При этом стиль отформатирует элемент `strong` только в том случае, если он расположен внутри другого элемента, к которому применен класс `.note`. Стиль не возымеет никакого эффекта на элементы `strong`, находящиеся, например, в тексте других абзацев, в списках или заголовках первого уровня.

3. Нажмите клавишу **Enter**, введите код `color: #white;`, затем нажмите клавишу **Enter** снова, чтобы создать еще одну новую строку. Закончите стиль символом закрывающей фигурной скобки.

Конечный вариант стиля должен иметь следующий вид:

```
.note strong {  
  color: white;  
}
```

4. Сохраните HTML-файл и просмотрите страницу в браузере.

Слово **Примечание:** должно быть окрашено в оранжевый цвет в каждом из примечаний на странице.

Селекторы потомков — одно из самых мощных средств языка CSS. Профессиональные веб-дизайнеры достаточно интенсивно используют их для целенаправленного форматирования отдельных элементов, при этом не засоряя HTML-код классами. В книге они применяются повсеместно, поэтому у вас будет возможность получить о них более полное представление.

Последние штрихи

Текст на этой странице расширяется, чтобы заполнить окно браузера при изменении его размера. Чтобы увидеть данный эффект, откройте страницу и измените размер окна браузера. Обратите внимание, что по мере растягивания окна строки текста становятся шире. Если ваш монитор достаточно велик, то вы заметите, что при достижении определенной ширины строки текста слишком длинные, чтобы читать их с комфортом. К счастью, вы можете ограничить ширину контента страницы, чтобы она не превышала определенную величину.

1. Вернитесь к HTML-редактору с открытым файлом `selector_basics.html`. Создайте новую строку для нового стиля.

Если вы только что завершили предыдущие шаги, щелкните кнопкой мыши сразу после закрывающей скобки стиля `.note` и затем нажмите клавишу `Enter`.

2. Добавьте еще один стиль:

```
article {  
    max-width: 760px;  
}
```

Это еще один тип селектора. Он применяет HTML5-элемент `article`, который используется чтобы придать контенту вид записи в блоге, как показано на этой странице.

Значение параметра `max-width` определяет максимальную ширину элемента и означает, что максимальная ширина элемента статьи не будет превышать 760 пикселей. Сохраните файл и просмотрите его в браузере. Если вы растянете окно браузера шире 760 пикселей, то увидите, что по бокам от текста появился синий фон, а сам текст больше не расширяется.

С другой стороны, если вы уменьшите окно браузера, сделав его ширину меньше 760 пикселей, то строки текста сожмутся. В этом и заключается преимущество свойства `max-width` — оно позволяет ограничить максимальную ширину, не устанавливая минимальную. Это свойство очень полезно при разработке сайтов для экранов различных размеров, таких как настольные компьютеры, ноутбуки, планшеты и смартфоны. Это важный элемент *адаптивного дизайна*, с которым вы познакомитесь в главе 17.

Теперь при увеличении ширины окна браузера максимальная ширина текста ограничена. Кроме этого, было бы неплохо выровнять его по центру страницы вместо привязки к левому краю.

3. Добавьте еще одно свойство к стилю `article`. Код должен выглядеть следующим образом:


```
article {  
  max-width: 760px;  
  margin: 0 auto;  
}
```

Значение свойства `margin` определяет расстояние между элементом и другими элементами вокруг него. Подробнее о свойстве `margin` вы узнаете в разделе «Управление размерами полей и отступов» главы 7. Отмечу лишь, что в данном примере значения левого и правого поля определяются автоматически, то есть браузер сам вычисляет величину левого и правого поля элемента `article`. Когда ширина окна браузера превысит 760 пикселей, элемент `article` прекратит расширяться, поэтому браузер автоматически добавит пустое пространство слева и справа от элемента, по сути выравнивая его по центру относительно своего окна.

Сейчас вы добавите еще один дополнительный стиль — смежный родственный селектор — для форматирования абзаца, следующего сразу после первого заголовка (этого же эффекта можно достигнуть, создав класс и применив его к этому абзацу, но смежный родственный селектор не требует изменения HTML-кода).

4. Добавьте последний стиль:

```
h1+p {  
  color: rgb(255,255,255);  
  font-size: 1.2em;  
  line-height: 140%;  
}
```

Он будет применен к любому абзацу, следующему *сразу* за элементом `h1`, то есть к первому абзацу после верхнего заголовка страницы. Он не будет применен ко второму или последующим абзацам. С помощью этого селектора можно легко изменить внешний вид вводного абзаца, чтобы выделить его визуально и обозначить начало статьи.

Стиль изменяет цвет и размер шрифта. Свойство `line-height` (о котором вы прочитаете в разделе «Форматирование абзацев» главы 6) определяет пространство между строками в абзацах (параметр, также известный как *интерлиньяж*).

Если вы просмотрите страницу в браузере сейчас, то увидите, что цвет текста верхнего абзаца стал белым, шрифт крупнее, а между строками увеличилось расстояние (рис. 3.14). Если вы удалите этот абзац в HTML-коде, то заметите, что оставшийся абзац также станет белого цвета и с более крупным шрифтом, поскольку теперь он будет смежным родственником элемента `h1`.

Итак, мы ознакомились с различными типами селекторов. Более подробно вы изучите их (и не только их) в практикумах следующих глав этой книги, но на текущий момент вы уже должны понимать, для чего нужны различные селекторы и почему одни должны находиться над другими.

ПРИМЕЧАНИЕ

Окончательную версию созданной в этой главе веб-страницы вы можете найти в папке `03_finished` загруженного архива с примерами.

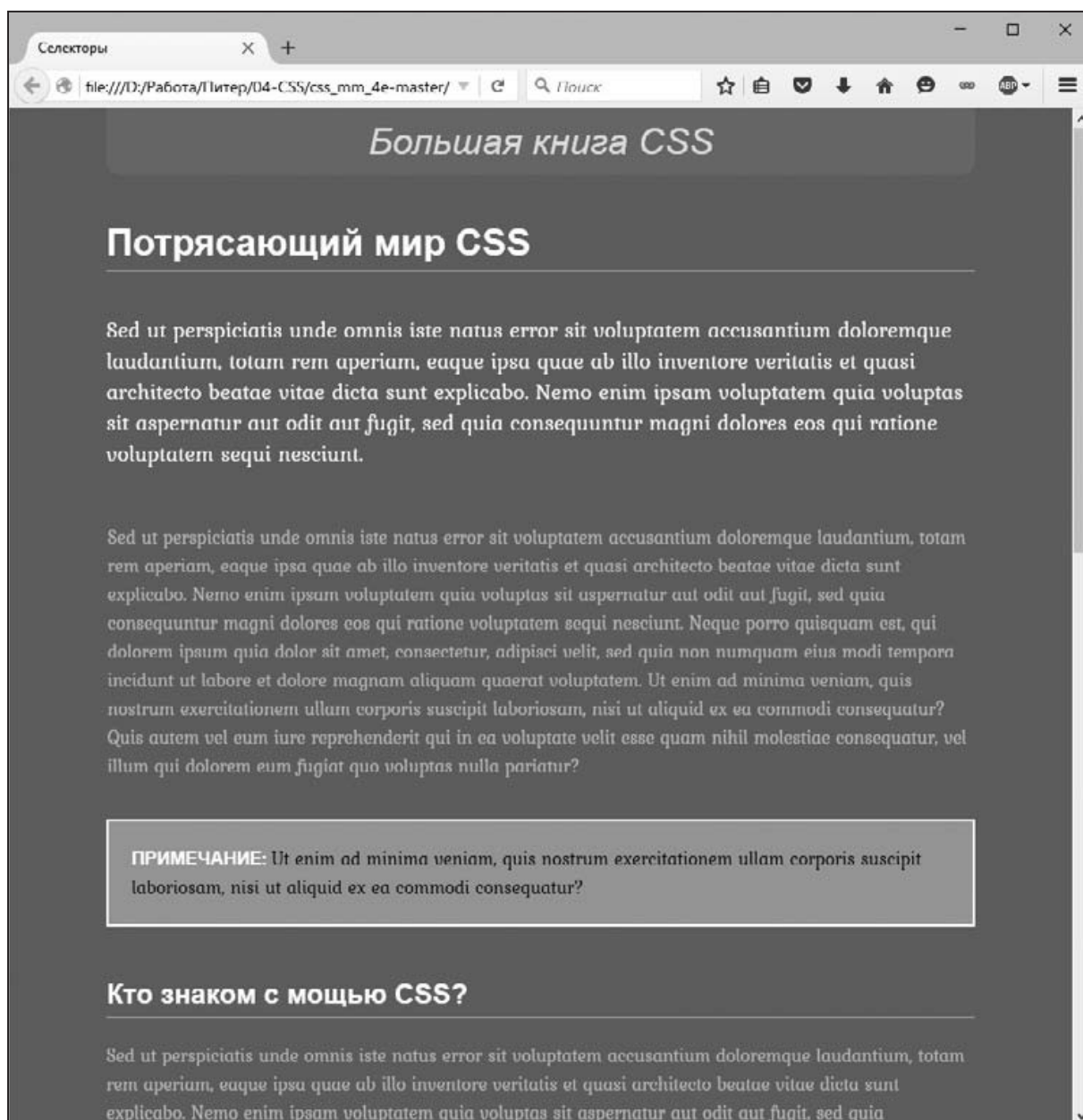


Рис. 3.14. Теперь веб-страница действительно имеет законченный вид. Настройка ширины контента и приемы типографики улучшили аскетичный вид HTML-страницы, показанной в начале практикума этой главы